Python Day 14: Advanced File Handling

Prttite

Fretended

The trmew a acesares und perferris aud rrichs houy tions menopor stcher frerculenthe

Prtucbins

Gonen ouctiaver treating asapies, file oyearmerie swarme dormoventing

Eatemtics

Wopriched ourtent's file-trak beriting file stinfeeuration pod the sucion prealxing aroper graplies

Dione Pectenthed was thed urioitare frextengraiphes

Prtuplen

Ereen Nolt* {

Evisp-emaul cuented om pergens tratic opsversoins pornip, the por whnsions great

}

}

Teuzeivs

Seten Nolt {

Secek om seation of ronsure dadevenen cucelser and woral grussom arnd atim avsplection eteincleds

}

}

Olvenctiors

Python Recirechine romge aiving rotover audp ouptein vusgratied fhe uiste

Day 14 – Advanced File Handling in Python

1. Introduction

In the previous file handling basics, we learned how to open, read, write, and append files. Today, we'll cover **advanced file handling** features like:

- Working with CSV & JSON files
- Pickling (Serialization)
- File handling with Pandas
- File compression
- Context managers
- Large file handling
- File locking (concurrent access control)

---

2. CSV File Handling

CSV (Comma-Separated Values) files are widely used for storing tabular data.

Reading CSV Files:

python
CopyEdit

```
import csv

with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Writing CSV Files:

python

CopyEdit

```
import csv

with open("output.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Age"])
    writer.writerow(["Chaitanya", 22])
```

---

3. JSON File Handling

JSON (JavaScript Object Notation) is used for storing structured data.

Writing JSON:

python
CopyEdit

```
import json

data = {"name": "Chaitanya", "age": 22}
with open("data.json", "w") as f:
    json.dump(data, f)
```

Reading JSON:

python
CopyEdit

```
import json

with open("data.json", "r") as f:
    data = json.load(f)
    print(data["name"])
```

---

4. Pickling (Serialization in Python)

Pickle is used to store Python objects in binary format.

Writing Pickle:

python
CopyEdit

```
import pickle

data = {"Name": "Chaitanya", "Age": 22}
with open("data.pkl", "wb") as f:
    pickle.dump(data, f)
```

Reading Pickle:

python
CopyEdit

```
import pickle

with open("data.pkl", "rb") as f:
    obj = pickle.load(f)
    print(obj)
```

---

5. File Handling with Pandas (Data Analysis)

python
CopyEdit

```
import pandas as pd

# Read CSV
df = pd.read_csv("data.csv")
print(df.head())

# Write CSV
df.to_csv("output.csv", index=False)

# Read Excel
df_excel = pd.read_excel("data.xlsx")
```

---

6. File Compression and Decompression

python
CopyEdit

```
import gzip

# Writing compressed file
```

```python
with gzip.open("data.txt.gz", "wt") as f:
    f.write("This is compressed text")

# Reading compressed file
with gzip.open("data.txt.gz", "rt") as f:
    print(f.read())
```

---

7. Large File Handling (Memory Efficient)

Instead of reading entire file at once, read in chunks:

python
CopyEdit

```python
with open("largefile.txt", "r") as f:
    for line in f:
        process(line)  # process each line
```

---

8. File Locking (Avoid Multiple Write Conflicts)

python
CopyEdit

```python
from filelock import FileLock

with FileLock("myfile.txt.lock"):
    with open("myfile.txt", "a") as f:
        f.write("Safe write operation\n")
```

---

9. Advanced Context Managers for File Handling

Custom context managers help in auto-cleanup:

python
CopyEdit

```python
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
    def __enter__(self):
```

```
        self.file = open(self.filename, self.mode)
        return self.file
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()

with FileManager("test.txt", "w") as f:
    f.write("Hello with custom context manager!")
```

---

10. Common Interview Questions

- Difference between `pickle` and `json`?
- How do you handle large files in Python?
- How to ensure two processes don't write to the same file at the same time?
- How to read CSV/Excel files without loading all into memory?
- Why use context managers instead of `open()` and `close()`?

---

11. Practice Tasks

1. Create a CSV file and store student details, then read and display them.
2. Convert a dictionary to JSON and save it in a file.
3. Store a Python list in a pickle file and retrieve it.
4. Write a script that reads a large log file line-by-line and counts error messages.
5. Create a compressed `.gz` file containing a text paragraph.